POF INSTITUTE OF TECHNOLOGY

Laravel Internship Report

Project:

Laravel REST API Development

Submitted By:

Intern: Mannahil Asad

Week: Week 4 (Day 16 – Day 20)

Department:

IT Department

Submitted To:

Mr. Babar

Session 2026

Week 4

Day 16: REST API Fundamentals & First API

Objective

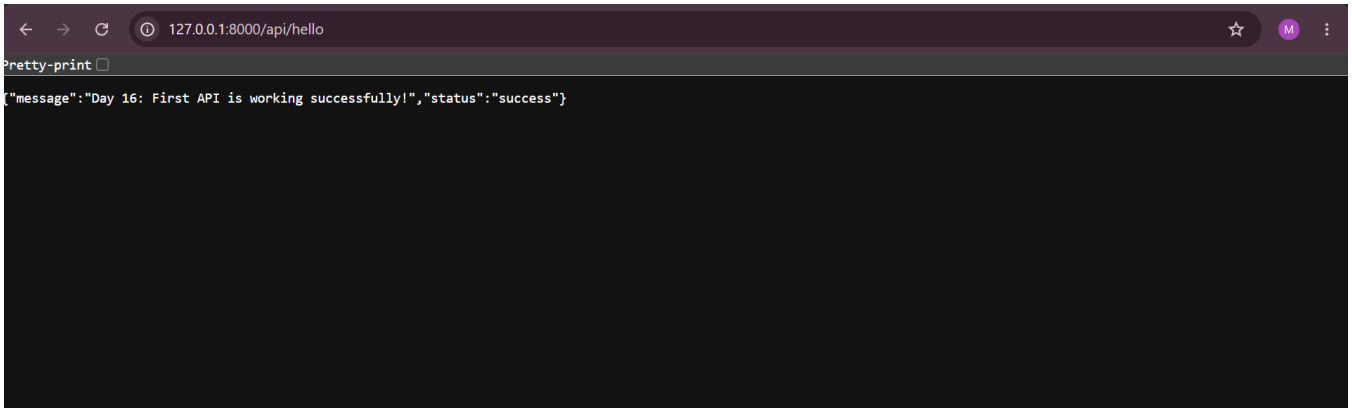
The objective of Day 16 was to understand the core concepts of RESTful architecture, differentiate between traditional web routes and API routes, learn standard HTTP methods, and successfully configure and test a basic Laravel API endpoint returning structured JSON data.

Step 1: Technical Work & Implementation Details

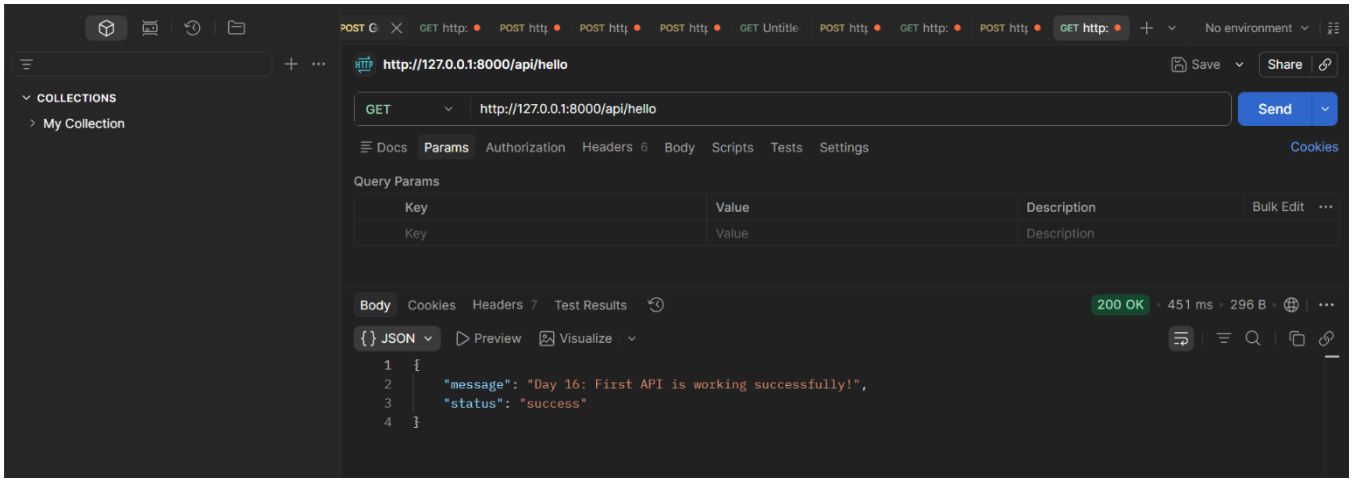
- **API Routing Setup:** Enabled API routing in the Laravel application using Artisan commands (php artisan install:api), which initialized the routes/api.php file and integrated Laravel Sanctum.
- **REST Architecture Concepts:** Explored how APIs handle stateless communication and transmit data in lightweight **JSON (JavaScript Object Notation)** format instead of rendering HTML views.
- **HTTP Methods Overview:** Studied the fundamental HTTP verbs used in API development:
- **First API Endpoint Development:** Created a custom GET route inside routes/api.php that returns a success message and status code in JSON format.

Step 2: Verification & Testing

- Started the local development server using php artisan serve.
- Accessed the endpoint via browser/tool at http://127.0.0.1:8000/api/hello and verified the correct JSON output.



Postman API Testing & Verification



Day 17: API CRUD Implementation with Validation

Objective

The objective of Day 17 was to implement a full set of CRUD (Create, Read, Update, Delete) operations for the Student resource within the Laravel API, ensuring robust data integrity through server-side validation.

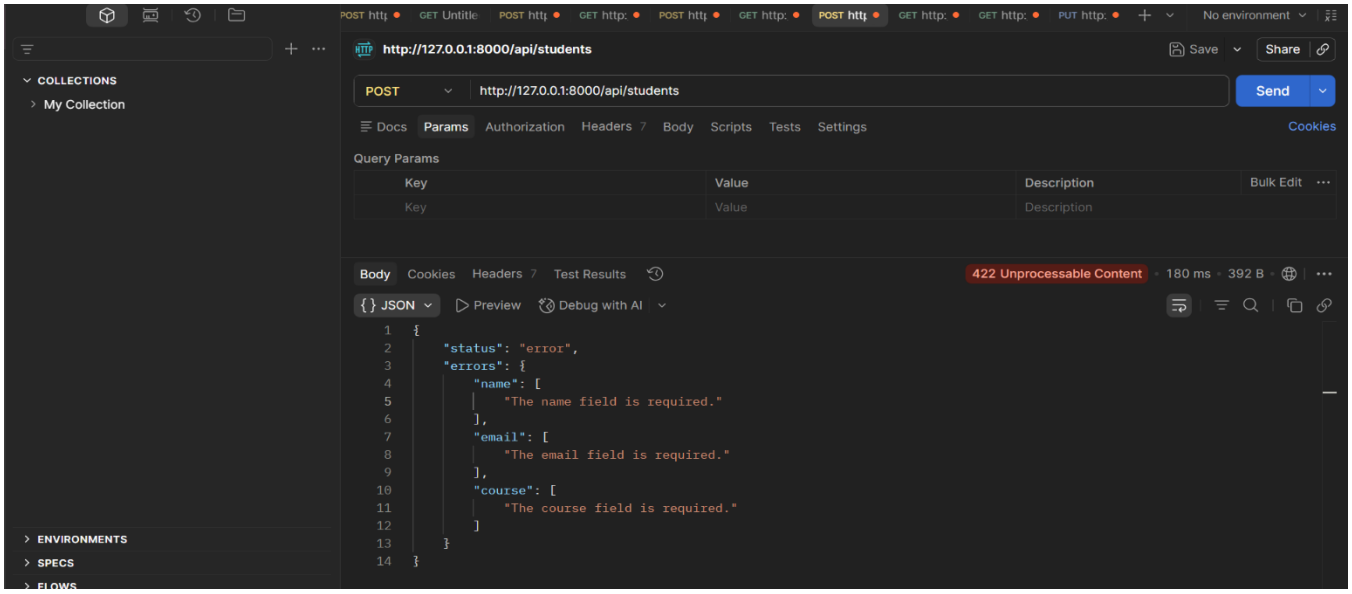
Step 1: Technical Implementation

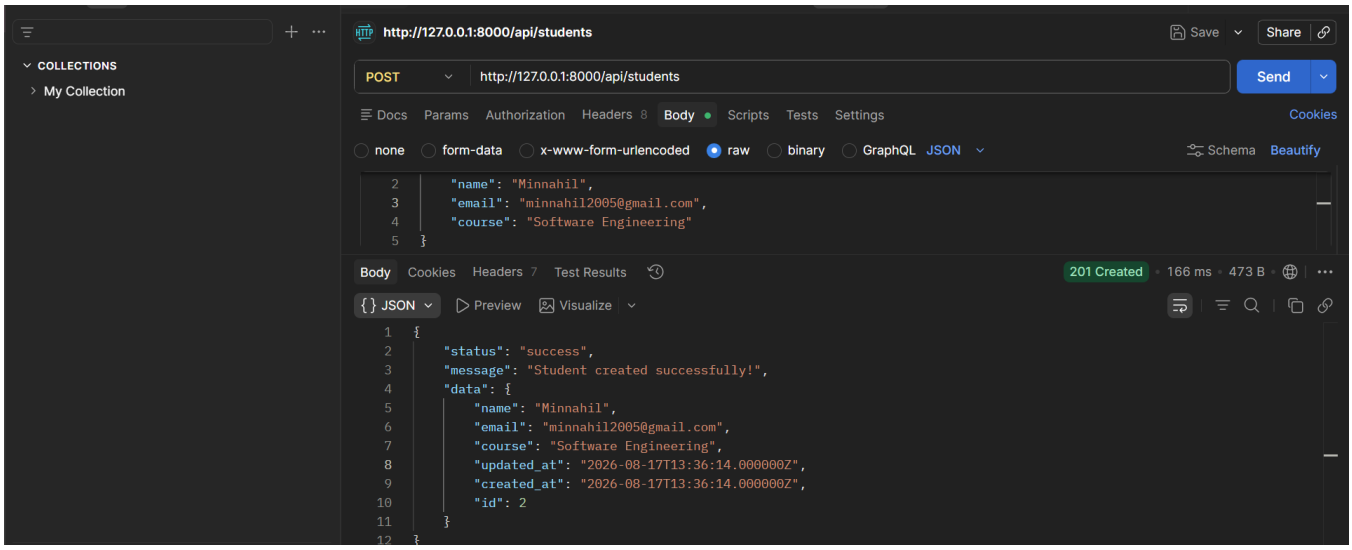
Created StudentController to manage HTTP requests.

- Added validation rules for mandatory fields and email uniqueness, returning 422 errors for invalid inputs.
- Implemented index, store, show, update, and destroy methods for database operations.
- Configured RESTful routes using Route::apiResource in api.php.
- Defined $fillable attributes in the Student model.

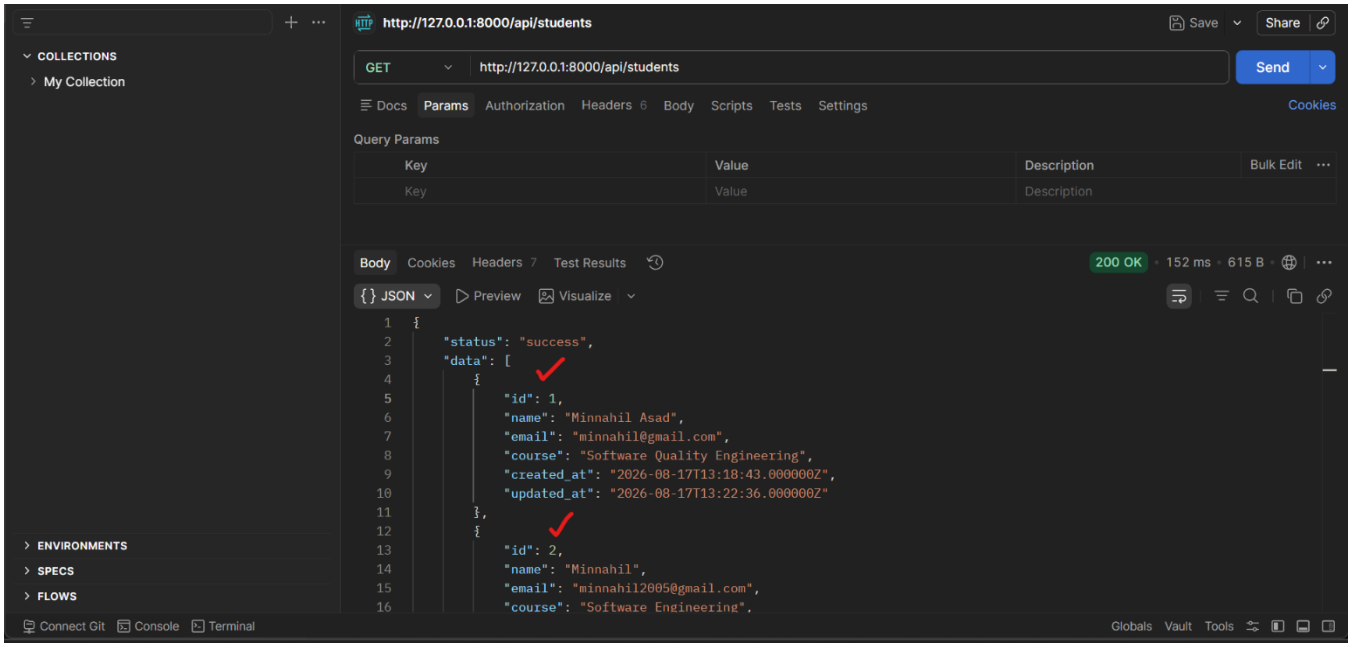
Step 2: API Testing (Postman)

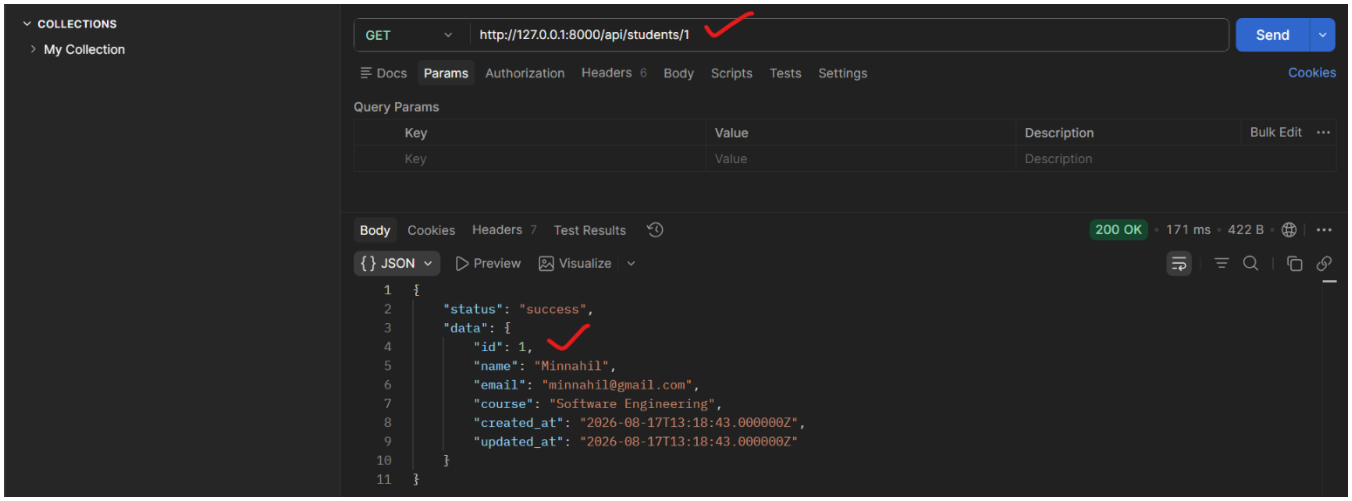
- **Create:**
  - ✓ **Validation Error (422):** Verified that submitting an empty request body correctly triggers server-side validation and returns a 422 Unprocessable Content status with required field messages.
  - ✓ **Successful Creation (201):** Confirmed that submitting valid student details via a POST request successfully stores the record and returns a 201 Created status response.



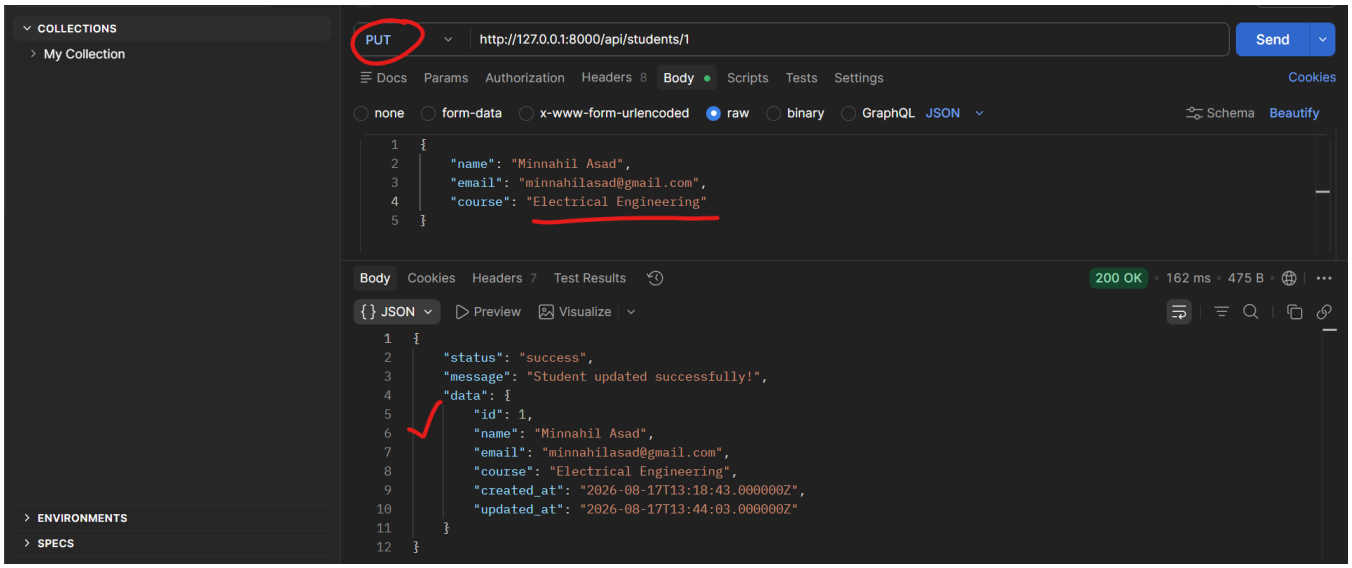


- **Read:**
  - ✓ **Get All Students:** Tested the `GET` request to retrieve the complete list of all saved student records from the database.
  - ✓ **Get Single Student:** Tested the `GET` request with a specific student ID (e.g., `/api/students/1`) to fetch individual student details.

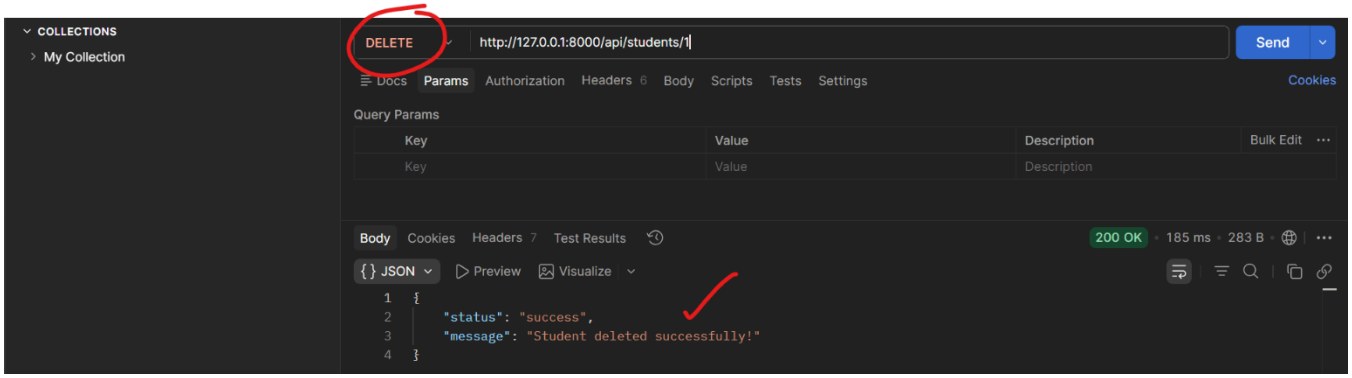




- **Update:** Confirmed successful modification of student details using PUT requests.



Delete: Verified record removal using DELETE requests with a success response

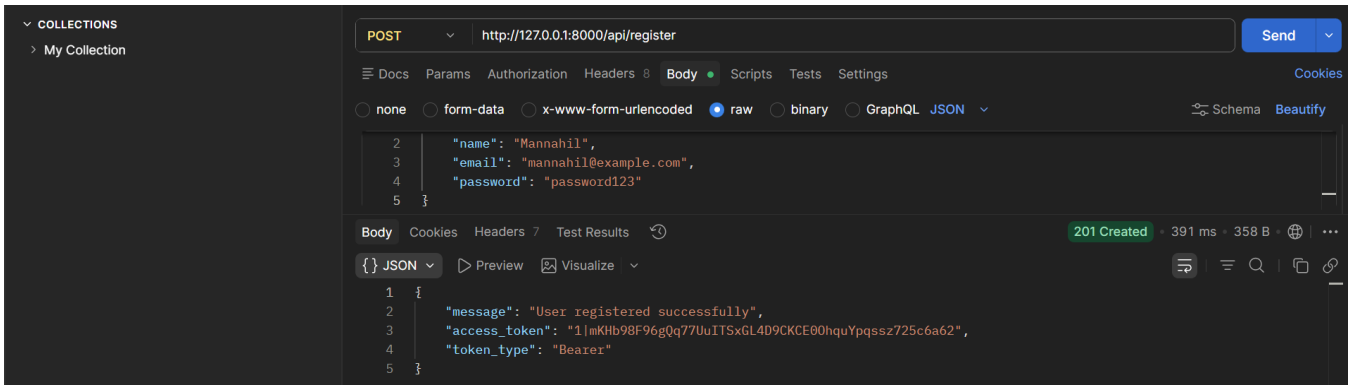


Day 18: API Authentication using Laravel Sanctum

Objective: To implement secure API authentication using Laravel Sanctum, generate API access tokens for registered users, and secure protected routes using bearer token authorization.

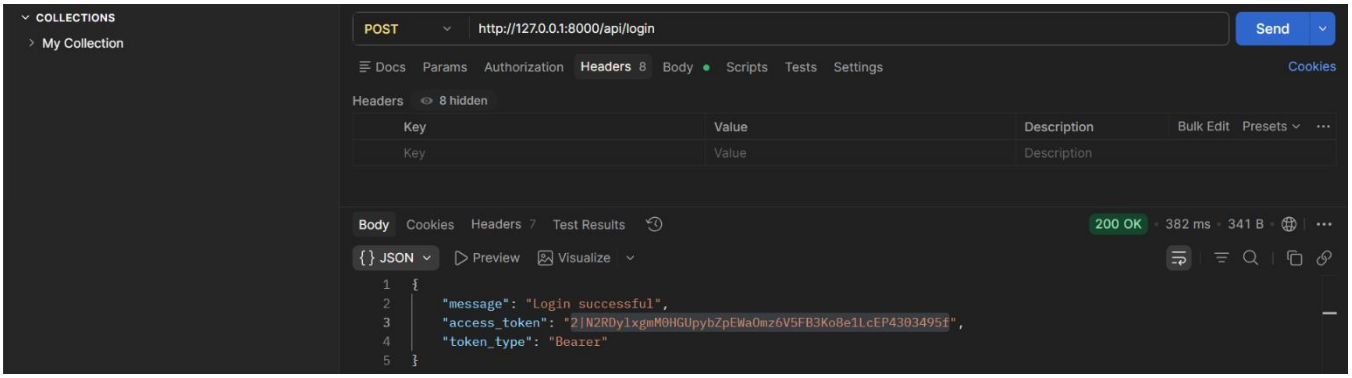
Step 1: User Registration API (POST /api/register):

Implemented and tested the registration endpoint to create new user accounts, returning a **201 Created** status along with a Sanctum bearer access token.



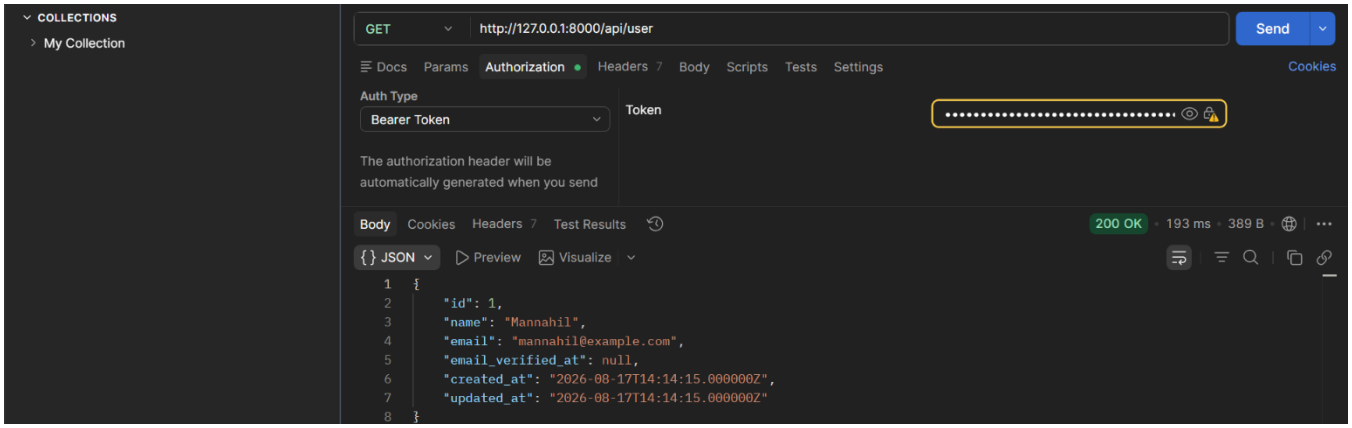
Step 2: User Login API (POST /api/login):

Tested the login endpoint using user credentials to authenticate sessions, returning a **200 OK** status and generating a secure access token.



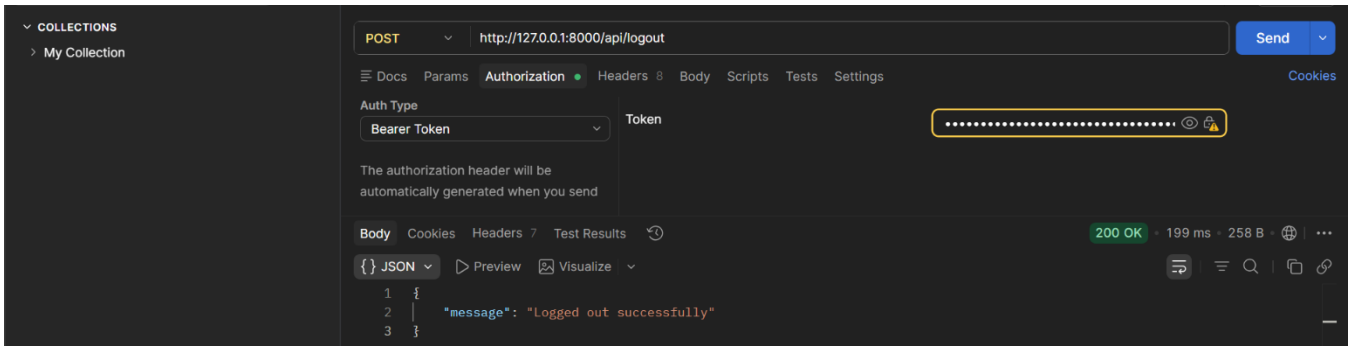
Step 3: Protected Routes (GET /api/user):

Verified token-based security and request authorization by passing the generated access token inside the **Bearer Token** authorization headers in Postman.



Step 4: User Logout API (POST /api/logout):

Tested the secure logout endpoint using token authorization to successfully revoke the active bearer token and terminate the user session.



Day 19: API Resources & Clean API Responses

Objective:

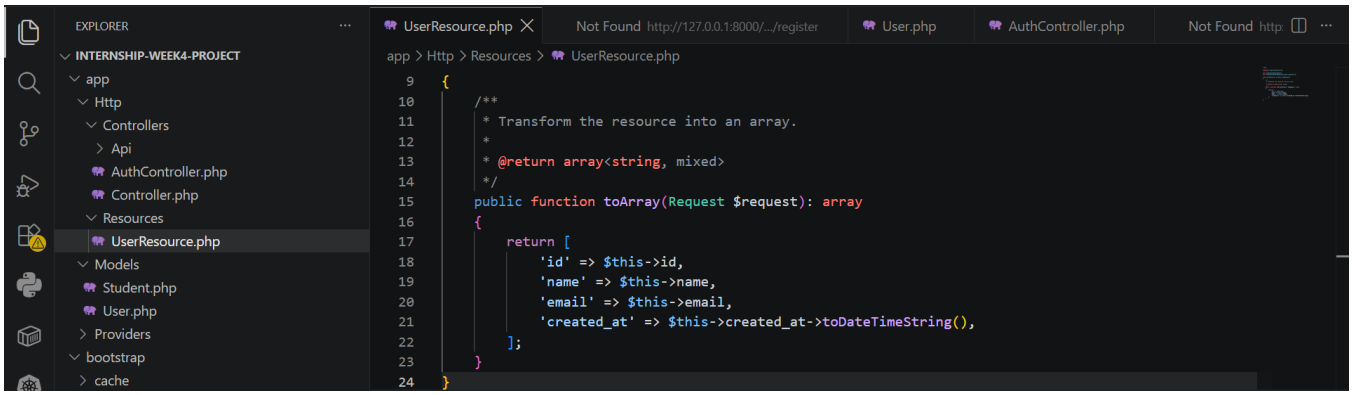
To transform raw database models into clean, structured JSON formats using Laravel API Resources while maintaining standard HTTP status codes.

API Resource Setup (UserResource):

Generated and configured UserResource to filter and return specific user attributes (id, name, email, and created_at).

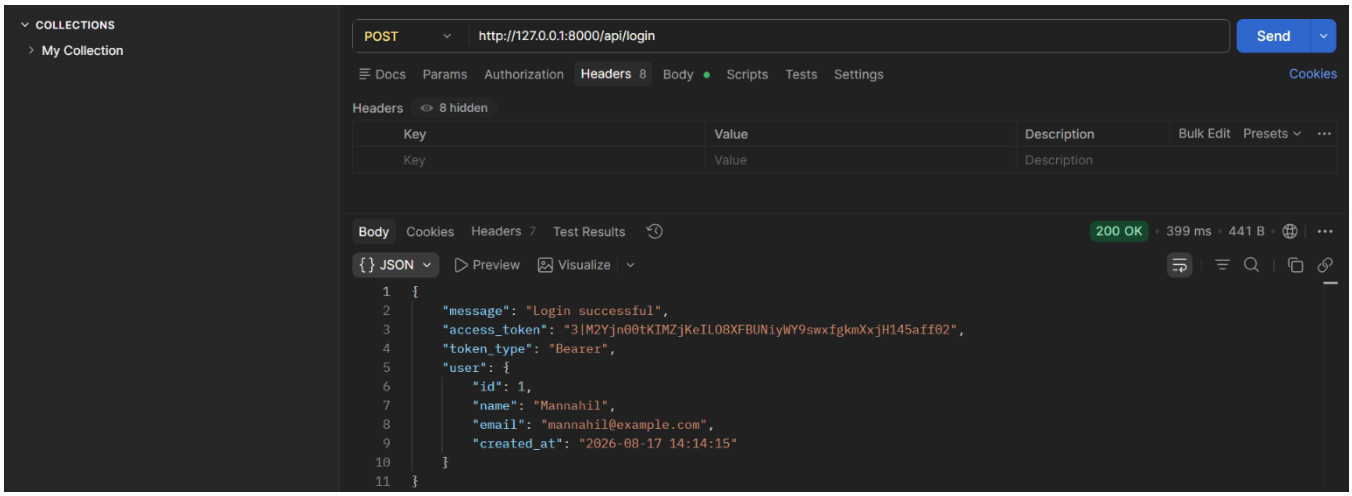
Controller Implementation:

Integrated UserResource into the registration and login responses within AuthController.



Postman Testing & Status Codes:

Verified the clean JSON output alongside proper HTTP status codes (200 OK and 201 Created) to ensure professional API communication.



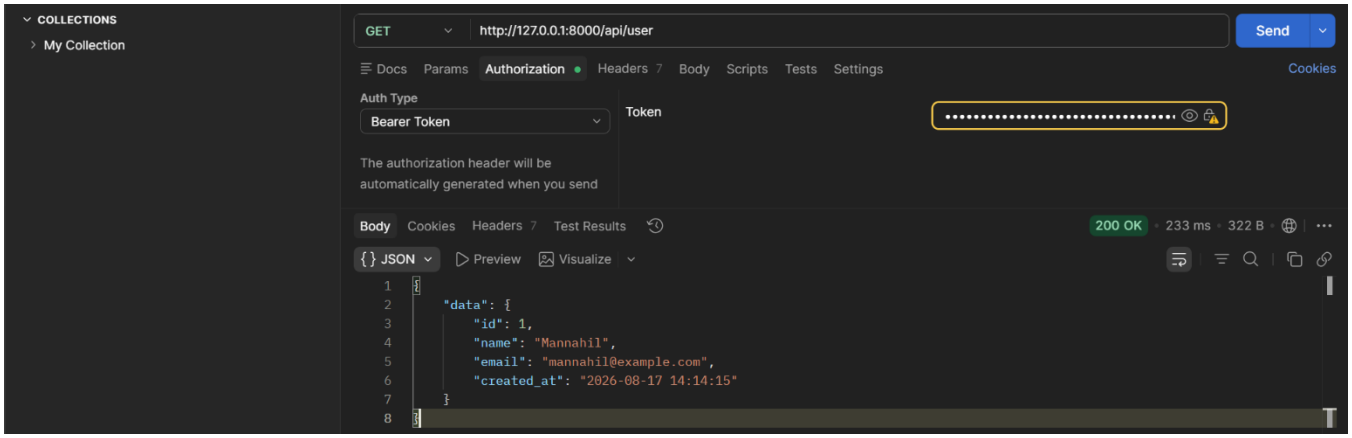
Day 20: Mini API Project & REST API Finalization

Objective:

The objective of Day 20 was to finalize the REST API project by implementing structured API resources, securing protected routes with Laravel Sanctum, and performing comprehensive functional testing to ensure professional API behavior.

Step 1: API Routes & Resources Finalization

Configured public and protected routes in routes/api.php and integrated UserResource to return clean user profile structures.



Step 2: User Registration & Token Testing

Tested the /api/register endpoint, ensuring proper validation rules, successful status codes (201 Created), and proper access token generation.

Step 3: Request Authorization & Error Handling/ Security / Unauthorized Access

Validated secure endpoints using Bearer Tokens and observed responses for unauthorized or malformed requests.